

MODULE 02 · DOMAIN 1 - FOUNDATION MODEL INTEGRATION, DATA MANAGEMENT, AND COMPLIANCE

Customising the Model

Task 1.2.4. Prompt, retrieve, fine-tune or distil, then deploy, version, roll back and retire.

DOMAIN 1 of 5 EXAM WEIGHT 31% MODULE 2 of 22

Customisation is a **ladder**. Every rung costs more money, more time and more maintenance than the one below it. Climb only as far as the requirement forces you.

The customisation ladder, cheapest first

01

Prompt

Instructions, examples, output format. Minutes to change.

02

Retrieve

RAG. Put the right facts in the context window.

03

Fine-tune

Labelled pairs teach a format, tone or task.

04

Pre-train

Unlabelled corpus teaches a domain's vocabulary.

ON THE EXAM

Training a model from scratch is a fifth rung and is explicitly out of scope for this exam. If an option offers it, that option is wrong.

What each rung actually fixes

SYMPTOM IN THE SCENARIO	THE RUNG THAT FIXES IT
Wrong format, rambling, ignores instructions	Prompt engineering
Does not know your products, policies or prices	RAG
Facts change daily or hourly	RAG. Nothing else keeps up
Right facts, wrong voice or wrong output shape	Fine-tuning
Does not understand your domain's jargon at all	Continued pre-training
Correct but too slow or too expensive	Distillation, or a smaller model

ON THE EXAM

Read the symptom, not the ambition. Scenarios often say the business 'wants a custom model' while describing a problem that RAG solves.

RAG or fine-tuning?

CHOOSE RAG WHEN

The problem is knowledge

- The model lacks facts it was never trained on
- Content changes and must stay current
- You need citations back to a source
- Access control differs per user

CHOOSE FINE-TUNING WHEN

The problem is behaviour

- You need a consistent tone or persona
- You need a rigid output structure every time
- The task is narrow and repeated at volume
- You have labelled examples of good output

THE TIEBREAKER

Knowledge problem, retrieve. Behaviour problem, fine-tune. And when both are true, do RAG first, because it is reversible and fine-tuning is not.

What fine-tuning does not do

- ◆ **It does not reliably add facts.** Training on documents teaches style far more readily than recall.
- ◆ **It does not stop hallucination.** It often makes the model more confidently wrong in your house style.
- ◆ **It does not stay current.** The day you finish training, the model begins going stale.
- ◆ **It cannot enforce per-user permissions.** Anything in the training data is available to every caller.
- ◆ It is genuinely excellent at one thing: making output **consistent**.

ON THE EXAM

The commonest wrong answer in this whole domain is fine-tuning a model on a document corpus to make it 'know' the documents. Distrust that option on sight.

A healthcare provider is building an internal assistant so staff can ask questions about clinical protocols. The protocols are revised by a committee roughly twice a month, and staff must be told which protocol version an answer came from.

Nurses may see all protocols; administrative staff may see only a non-clinical subset. The provider's data science team has proposed fine-tuning a foundation model on the full protocol library so that the assistant responds instantly and without retrieval infrastructure.

How should the solution be built?

- ☐ A Fine-tune the model on the protocol library and retrain after each committee revision
- ☐ B Use RAG over the protocol library with metadata filtering by staff role, returning source citations
- ☐ C Use continued pre-training on the protocol library to teach the model clinical vocabulary
- ☐ D Fine-tune two separate models, one for clinical staff and one for administrative staff

◆ PAUSE AND DECIDE

How should the solution be built?

- A** Fine-tune and retrain after each revision
Fails all three requirements. No citations, no per-role access, and a retraining cycle every fortnight for content that fine-tuning does not reliably memorise anyway.
- B** RAG with metadata filtering and citations
Handles freshness by re-indexing, citations by returning the retrieved chunk's source, and access control by filtering on a role attribute at query time.
- C** Continued pre-training
Teaches vocabulary, not recall. It is the most expensive rung on the ladder and satisfies none of the three stated requirements.
- D** Two fine-tuned models
Solves access control by brute force, then doubles the retraining bill every fortnight and still provides no citations.

THE SIGNAL

Three requirements, all three fatal to training: content changes fortnightly, answers need source attribution, and permissions differ by role. Any one of them alone rules out fine-tuning.

Five ways to get a customised model on Bedrock

MECHANISM	WHAT YOU SUPPLY	BEST FOR
Fine-tuning	Labelled prompt-response pairs	Tone, format, narrow repeated tasks
Continued pre-training	Unlabelled domain corpus	Domain vocabulary and patterns
Model Distillation	Prompts only, plus a teacher	Cost and latency, similar accuracy
Custom Model Import	Your own model weights	Reusing work done elsewhere
SageMaker AI, then import	Full control of the training loop	Techniques Bedrock will not expose

ON THE EXAM

Custom Model Import takes open architectures such as Llama and Mistral, and imported models can be invoked on demand. It cannot import proprietary models.

Model Distillation, and why it is not fine-tuning

- ◆ A large **teacher** model generates responses. A small **student** model is trained to reproduce them.
- ◆ **You supply prompts only.** Bedrock generates the responses. No labelled dataset to write.
- ◆ It can read your **model invocation logs** as the prompt source, skipping synthetic generation entirely.
- ◆ Goal: a small model's cost and latency at something near the large model's accuracy, **for one narrow use case**.
- ◆ The student is worse than the teacher at everything you did not distil for. That is the trade.

ON THE EXAM

The trigger phrase is 'similar accuracy at lower cost and latency'. Distillation is the only mechanism whose stated purpose is that sentence.

Distillation or fine-tuning?

DISTILLATION

You have prompts and a good big model

- Supply prompts. Bedrock makes the labels
- Goal is cost and latency at similar quality
- Teacher's behaviour is the target
- Invocation logs can seed it for free

FINE-TUNING

You have labelled examples of correct output

- Supply prompt and response pairs yourself
- Goal is a behaviour no model does yet
- Your labelled data is the target
- Needs a curated, diverse dataset

THE TIEBREAKER

Do you already have a model producing output you like? Distil it. Do you have humans producing output you like? Fine-tune on that.

PEFT, LoRA and adapters

- ◆ **Full fine-tuning** updates every weight. Expensive to train, and you now host a whole new model.
- ◆ **PEFT** freezes the base model and trains a small number of new parameters instead.
- ◆ **LoRA** is the common PEFT method: low-rank matrices injected alongside the frozen weights.
- ◆ Adapters are **small**. Many task-specific adapters can share one hosted base model.
- ◆ On Bedrock, PEFT-customised models are eligible for **on-demand** deployment. Full-rank customisations often are not.

ON THE EXAM

Skill 1.2.4 names LoRA and adapters explicitly. Recognise them as the cheap, swappable alternative to producing an entire new model per task.

An e-commerce company classifies incoming support emails into 40 categories using a frontier model on Amazon Bedrock. Accuracy is excellent and the business is satisfied with the classifications.

Volume has grown to 900,000 emails a day. The model spend is now the largest line in the platform budget and P95 latency has become a problem for the routing system downstream. Twelve months of model invocation logs containing prompts and the accepted classifications are already stored. The team must cut cost and latency without a measurable drop in classification accuracy.

What should the team do?

- | | |
|---|---|
| <p>A Run a Bedrock Model Distillation job using the invocation logs, with the frontier model as teacher and a small model as student</p> | <p>B Switch directly to the smallest available model and adjust the prompt to compensate</p> |
| <p>C Apply continued pre-training to a small model using the twelve months of email text</p> | <p>D Purchase Provisioned Throughput for the frontier model to reduce the per-token rate</p> |

What should the team do?

- A

 Model Distillation seeded from invocation logs
Exactly its purpose. The logs hold prompts and accepted responses, so the teacher never has to regenerate them, and the student inherits behaviour on this one narrow task.
- B

 Smallest model plus prompt tuning
Cheapest and fastest, and abandons the accuracy constraint. A small model prompted cold will not match a frontier model on a 40-way classification.
- C

 Continued pre-training on email text
Teaches the model what emails sound like. The model already understands emails. The task it must learn is the 40-category mapping, which needs labels.
- D

 Provisioned Throughput for the frontier model
Changes the billing shape, not the unit economics, and does nothing at all for latency. The expensive model is still doing the work.

THE SIGNAL

Two phrases decide it: 'without a measurable drop in accuracy' and 'logs containing prompts and the accepted classifications'. The second sentence is the dataset being handed to you.

Three ways to serve a customised model

PATH	HOW IT BILLS	CHOOSE IT WHEN
Provisioned Throughput	Model units, hourly, always on	Steady load, or the customisation is full-rank
Custom model deployment	On demand, per token	Variable or low volume. Cold start on first call
SageMaker AI endpoint	Instance hours	You need instance control, or the model is not on Bedrock

ON THE EXAM

On-demand custom model deployment is newer and narrower than Provisioned Throughput. It depends on model, customisation method and Region, so verify eligibility rather than assuming.

Older material says a custom model can **only** run on Provisioned Throughput. That was true, and is now only mostly true.

SageMaker Model Registry, and why the exam likes it

- ◆ A **model package group** holds versions of one logical model. Every training run registers a new version.
- ◆ Each version carries an **approval status**: pending, approved or rejected. That status is the deployment gate.
- ◆ A pipeline can require **manual approval** before deployment, which is the auditable human checkpoint.
- ◆ Versions carry **metadata and lineage**: training data, metrics, and the model card.
- ◆ Rollback becomes trivial: **redeploy the previous approved version**. It is still sitting there.

ON THE EXAM

Whenever a scenario wants governed, repeatable, auditable model promotion, Model Registry plus an approval gate is the shape of the answer.

Automated deployment for a customised model

01

Train

Customisation job runs on new data.

02

Evaluate

Score against a held-out golden dataset.

03

Register

Version lands in the registry as pending.

04

Approve

Quality gate passes, or a human signs off.

05

Deploy

Canary a slice of traffic, then widen.

ON THE EXAM

The evaluate step before register is what makes the gate meaningful. A pipeline that registers and approves automatically without scoring has a gate in name only.

Canary or shadow?

CANARY

Real users, small slice

- A percentage of live traffic hits the new version
- Those users see the new output
- Roll back on an alarm and widen on success
- Fastest real signal, some blast radius

SHADOW

Real traffic, no users

- Traffic is mirrored to the new version
- Its responses are logged and discarded
- Users are never exposed to the new model
- Zero blast radius, no user feedback either

THE TIEBREAKER

'Must not affect any customer during validation' means shadow. 'Limit the impact if it goes wrong' means canary. The wording distinguishes them precisely.

A bank retrains a fine-tuned model monthly as new regulatory guidance is published. Deployments are currently manual, and last month an untested version reached production and produced malformed output for six hours before anyone noticed.

Compliance now requires that no model version reaches production without a recorded human approval, that the reviewer can see the evaluation scores at the moment of approval, and that reverting to the previous version takes minutes rather than a retraining cycle.

Which approach meets the compliance requirements?

- ☐ A Register each version in SageMaker Model Registry with evaluation metrics attached, gate the pipeline on a manual approval step, and roll back by redeploying the last approved version
- ☐ B Add an automated quality gate that blocks deployment if evaluation scores fall below a threshold
- ☐ C Store each model artefact in Amazon S3 with versioning enabled and require a pull request before deployment
- ☐ D Deploy every version behind a shadow endpoint for one week before promoting it

Which approach meets the compliance requirements?

- A

 Model Registry, metrics, manual approval, redeploy to roll back
Maps one to one onto the three requirements: approval status is the record, attached metrics inform the reviewer, and the previous approved version is still registered.
- B

 Automated quality gate on thresholds
Good engineering and would have caught last month's failure. But compliance asked for a recorded human approval, and an automated gate is not a human.
- C

 S3 versioning plus a pull request
Versions artefacts and records an approval, but the reviewer sees a file, not evaluation scores. Rollback is manual redeployment with no lineage.
- D

 One-week shadow deployment
Validates behaviour well and addresses none of the three requirements. No approval record, no metrics at approval time, no faster rollback.

THE SIGNAL

Read compliance requirements as a checklist and tick them off per option. B is the better engineering answer and the wrong exam answer, because 'recorded human approval' is explicit.

Retiring a custom model

- ◆ Every custom model version incurs a **monthly storage charge**. Abandoned experiments bill forever.
- ◆ **Provisioned Throughput bills whether or not you call it**. A forgotten deployment is pure waste.
- ◆ When the base model reaches **Legacy**, your customisation of it inherits that clock.
- ◆ Re-evaluate periodically. A newer base model with a good prompt often beats last year's fine-tune.
- ◆ Retirement plan: measure the successor, shadow it, canary it, then delete the old version and its deployment.

ON THE EXAM

Skill 1.2.4 ends with 'lifecycle management to retire and replace models'. Retirement is an examinable step, not an afterthought.

A company fine-tuned a model eighteen months ago to produce structured product descriptions in a specific format. It runs on Provisioned Throughput and costs a fixed amount each month regardless of load.

The base model it was built on has just moved to Legacy status with an end-of-life date in nine months. A junior engineer has been asked to plan the response and proposes immediately starting a new fine-tuning run on the newest available base model, reusing the original training dataset.

What should the team do first?

- ☐ A Start the fine-tuning run immediately, since nine months is a short window for a regulated deployment
- ☐ B Evaluate the newest base model with prompt engineering against the same golden dataset before deciding to fine-tune at all
- ☐ C Request an extension of the end-of-life date from AWS Support
- ☐ D Migrate the existing fine-tuned model to an on-demand custom model deployment to reduce cost while planning

What should the team do first?

A Start the fine-tuning run immediately
Assumes the answer. It recreates a maintenance obligation and a Provisioned Throughput bill that may no longer be necessary.

C Request an EOL extension
EOL dates are published service lifecycle decisions. This defers a migration that has to happen anyway.

B Evaluate the new base model with prompting first
The dataset is already a golden dataset. Testing whether a plain prompt now clears the bar costs days and may remove the custom model entirely.

D Migrate to on-demand deployment
A sensible cost move on a model that is going away in nine months. It optimises something you are about to delete.

THE SIGNAL

The word 'first' is doing the work. Model generations improve quickly, and structured output is exactly what newer models got better at. Measure before you rebuild.

Bedrock or SageMaker AI?

- ◆ **Bedrock** when you want a managed, API-driven job with few hyperparameters and no infrastructure.
- ◆ **SageMaker AI** when you need control of the training loop, or a technique Bedrock does not expose.
- ◆ **SageMaker JumpStart** for open models with prebuilt fine-tuning recipes and one-click deployment.
- ◆ The exam signal is the persona: **application builder** means Bedrock, **data scientist or ML engineer** means SageMaker AI.
- ◆ You can train in SageMaker AI and import the result into Bedrock to use guardrails and knowledge bases with it.

THE TIEBREAKER

AWS's own guidance splits these by who is doing the work, not by what the model is. Scenarios describe the team for a reason.

A legal technology startup wants its assistant to write contract clauses in the firm's precise house style. Partners have collected 4,000 reviewed example clauses, each paired with the instruction that produced it. Style consistency is the entire product, and the clause library changes rarely.

The team has two engineers, no ML specialists, and no existing training infrastructure. They want the customised model integrated with Bedrock Guardrails and a knowledge base of case law. Prompt engineering with twenty examples was tried and produced inconsistent voice across long documents.

Which approach fits the requirement and the team?

- | | |
|---|--|
| ☐ A Fine-tune a Bedrock model on the 4,000 pairs using a Bedrock customisation job | ☐ B Use RAG over the 4,000 example clauses so the model retrieves similar clauses as style references |
| ☐ C Run continued pre-training in SageMaker AI on the firm's full document archive | ☐ D Train a model in SageMaker AI with a custom loop, then import it into Bedrock |

Which approach fits the requirement and the team?

- A** Bedrock fine-tuning job on the 4,000 pairs
A behaviour problem, labelled pairs already in hand, stable content, and a team with no ML specialists. Bedrock's managed job is built for exactly this shape.
- B** RAG over the example clauses
Retrieval supplies facts well and style poorly. Prompting with examples already failed on long documents, and RAG is that same technique with better example selection.
- C** Continued pre-training on the archive
Unlabelled training when they hold 4,000 labelled pairs. It discards the most valuable asset they have and costs far more.
- D** Custom SageMaker loop, then import
Technically capable and wrong for two engineers with no ML background. It buys control they cannot use and infrastructure they must maintain.

THE SIGNAL

This is the mirror of scenario 1. There, three signals forbade fine-tuning. Here, four permit it: behaviour not knowledge, labelled pairs in hand, stable content, and a team that needs it managed.

The five things to carry forward

- ◆ Climb the **ladder** as little as possible. Prompt, then retrieve, then fine-tune, then pre-train.
- ◆ **Knowledge problem, retrieve. Behaviour problem, fine-tune.** Fine-tuning does not add facts, stop hallucination, or respect permissions.
- ◆ **Distillation** is the answer to "same accuracy, less cost and latency". Prompts only, and invocation logs can seed it.
- ◆ **Model Registry plus an approval gate** is the shape of every governed deployment answer. Rollback is redeploying the last approved version.
- ◆ **Re-evaluate before you rebuild.** A new base model with a good prompt often beats last year's fine-tune.

ON THE EXAM

Module 3 moves to Task 1.3: getting data into a state a foundation model can actually consume. Documents, audio, images and the pipelines that clean them.